

JAVA LAB WEEK 11

Title: Mobile Number Validation Using Exception Handling

1. Objective / Aim

To write a Java program to check whether a mobile number is valid or invalid using exception handling.

The mobile number must contain exactly 10 digits from 0 to 9. The program demonstrates try, catch, throw, finally, built-in exceptions, and a user-defined exception.

2. System Requirements

Hardware Requirements

- Computer or laptop
- Minimum 2 GB RAM

Software Requirements

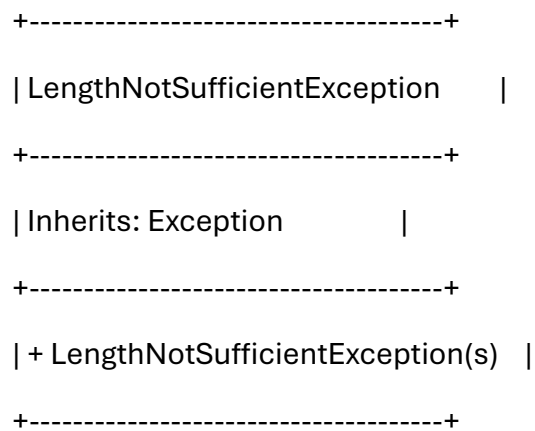
- Operating System: Windows / Linux
- JDK: Java 8 or above
- IDE: Eclipse, IntelliJ IDEA, NetBeans, or Dev-C++ with Java support (or any Java-compatible editor and terminal)

3. Algorithm / Logic Description

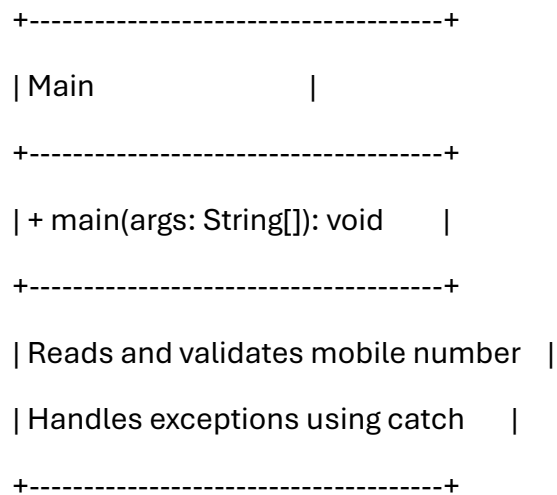
1. Start the program.
2. Import the Scanner class.
3. Create a user-defined exception named LengthNotSufficientException.
4. Read a mobile number from the user as a string.
5. Use a try block to validate the number.
6. If the number contains any non-digit character, throw NumberFormatException.
7. If the length is greater than 10, throw ArrayIndexOutOfBoundsException.
8. If the length is less than 10, throw LengthNotSufficientException.
9. If all conditions are satisfied, display **Valid number**.
10. Use separate catch blocks to handle the exceptions.
11. Use the finally block to close the scanner.

12. Stop the program.

4. UML Class Diagram



Used by



5. Source Code

Filename: Main.java

```
import java.util.Scanner;
```

```
class LengthNotSufficientException extends Exception {
    LengthNotSufficientException(String s) {
```

```
        super(s);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter mobile number: ");  
        String n = sc.nextLine();  
  
        try {  
            if (!n.matches("[0-9]+"))  
                throw new NumberFormatException();  
  
            if (n.length() > 10)  
                throw new ArrayIndexOutOfBoundsException();  
  
            if (n.length() < 10)  
                throw new LengthNotSufficientException("Invalid");  
  
            System.out.println("Valid number");  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Invalid Mobile Number-ArrayIndexOutOfBoundsException");  
        }  
        catch (LengthNotSufficientException e) {  
            System.out.println("Invalid Mobile Number – LengthNotSufficientException");  
        }  
    }  
}
```

```
        catch (NumberFormatException e) {  
            System.out.println("Invalid Mobile Number – NumberFormatException");  
        }  
        finally {  
            sc.close();  
        }  
    }  
}
```

6. Compilation and Execution Commands

Save the program as Main.java.

Compilation:

```
javac Main.java
```

Execution:

```
java Main
```

7. Output Screen

Output 1: Valid Mobile Number

Enter mobile number: 9885089465

Valid number

Output 2: Number Exceeds 10 Digits

Enter mobile number: 98567890121

Invalid Mobile Number-ArrayIndexOutOfBoundsException Exception

Output 3: Number Contains Less Than 10 Digits

Enter mobile number: 88664433

Invalid Mobile Number – LengthNotSufficientException

Output 4: Number Contains Non-Digit Characters

Enter mobile number: 98ab@123

Invalid Mobile Number – NumberFormatException

8. Result

Thus, the Java program for mobile number validation was successfully implemented using exception handling. The program demonstrates try, catch, throw, finally, built-in exceptions, and a user-defined exception to validate a mobile number.